

Financial Transaction Category Classification

Aleksa Ocampo, Alisa Crowe, Angela Lee, Brianna Sengchan
CS 549 – Spring 2026

Abstract:

This project focused on classifying personal financial transactions into predefined categories using supervised machine learning techniques. We utilized the Personal Transactions dataset, which includes features such as transaction descriptions, dates, account types, and monetary amounts. After preprocessing the dataset through feature engineering, encoding, and scaling, we implemented four different classification models: Logistic Regression, Random Forest, Support Vector Machine (SVM), and K-Nearest Neighbors (KNN).

The models were evaluated using precision, recall, and F1-score to account for class imbalance. Our results demonstrate that Random Forest achieved the strongest overall performance, while also highlighting the trade-offs between accuracy, interpretability, and computational efficiency. This work contributes to automated financial categorization systems, which are useful for budgeting, expense tracking, and personal financial planning.

1. Introduction:

1.1 Background

Financial transaction classification plays a key role in personal finance management by enabling automated budgeting, spending analysis, and financial planning. With the increasing volume of digital transactions in the modern world, manual categorization becomes inefficient and easily prone to errors. Machine learning provides an effective approach to automatically classify transactions based on patterns in historical data.

1.2 Problem Statement

The objective of this project was to develop a machine learning system capable of accurately classifying financial transactions into predefined categories using structured features such as transaction descriptions, dates, account information, and transaction amounts.

1.3 Dataset Overview

We used the Personal Transactions dataset from Kaggle, which contains both numerical and categorical features including Date, Description, Amount, Account Name, and Transaction Type. The target variable is the transaction Category.

Although we initially considered combining this dataset with an additional personal finance dataset, we chose to exclude it due to inconsistencies in the Description field, which contained non-meaningful or

randomly-generated text. Since transaction descriptions are a critical feature for classification, using inconsistent data would likely reduce model performance.

2. Methodology:

2.1 Data Preprocessing

Data preprocessing for the Personal Transactions dataset consisted of several steps, including restructuring the target variable column, handling null values, reformatting and featurization of dates, categorical column encoding, numerical column scaling, and feature engineering. The data cleaning code can be found in `data_cleaning.ipynb`.

After the initial exploration of the dataset, it became evident that a large portion of the dataset contained what appeared to be inconsistent or seemingly random values for the **Category** column, as shown in Figure 1.1 below. To address this issue, we decided to use the **Description** category to automatically map each row to five distinct categories: Bills & Payments (3,820 rows), Food & Dining (3,725 rows), Shopping, Personal & Entertainment (2,330 rows), Income (673 rows), and Transportation (258 rows). This approach enabled the models to learn meaningful and predictive patterns from the data, which was difficult to achieve when relying on the original randomly generated labels.

	Description	Category
1723	Brunch Restaurant	Paycheck
9136	Mortgage Payment	Home Improvement
4129	Biweekly Paycheck	Credit Card Payment
7728	Thai Restaurant	Credit Card Payment
780	Credit Card Payment	Credit Card Payment
431	Grocery Store	Groceries
1020	Amazon	Gas & Fuel
7646	American Tavern	Groceries
8656	Grocery Store	Restaurants
3204	Brewing Company	Credit Card Payment

Figure 1.1: Random Sample of Original Description vs. Category Columns

Next, because the **User ID** column contained 10,000 null values, this column was dropped entirely as it was not necessary to analyze overall spending patterns across customers. The dataset contained no duplicate values and no additional null values besides **User ID**. Additionally, the **Date** column was reformatted from the original pattern of 'M/D/YYYY' to a standard datetime format, which allowed for extraction of components such as **Day of Week** and **Month**. These two new features were created to help us analyze spending patterns over time: for example, we wanted to see if the Entertainment category is higher on the weekends or if Shopping increases in December. These features were both kept as integers, with values for **Day of Week** ranging from 0 to 6 and **Month** ranging from 1 to 12. Because both features have natural ordinal relationships, a simple ordinal encoding built into datetime logic was enforced.

With the basic cleaning completed, we took the dataset and counted how many unique descriptions were repeated. This allowed for us to manually sort and organize target words for each description into its corresponding category. However, we realized that focusing only on the **Description** column resulted in synthetically high accuracies because the models trained with this feature were essentially deterministic. In order to fix this issue, we decided to remove this column from the shared preprocessing pipeline and instead train the models on the other features.

Next, the categorical columns were encoded as follows:

- **Transaction Type**: Binary label encoding mapped the values “credit” → 0 and “debit” → 1.
- **Account Name**: One-hot encoding was selected for the three distinct values (Platinum Card, Checking, and Silver Card) to preserve the non-ordinal nature of these values.
- **Category (Target)**: Label encoding was used for the classes of the target variable for the models to treat as a distinct class ID rather than an ordinal relationship.

The numerical feature column **Amount** was scaled using StandardScaler with a mean of 0 and a standard deviation of 1. This choice was made because this column originally contained a wide range of values up to 2000+. Without scaling, the larger values would have dominated the feature space, which contained many 0's and 1's from one-hot encoding categorical columns.

Finally, feature engineering was implemented to introduce interaction features that could capture more complex relationships between variables. A binary **Is_Weekend** feature was created from the **Day of Week** column, where values of 5 and 6 (Saturday and Sunday) were mapped to 1, and all other days to 0. This feature was designed to help the models identify any behavioral differences in spending between weekdays and weekends. Additionally, an **Amount x Transaction Type** feature was created by multiplying the standardized **Amount** column with the encoded **Transaction Type** to differentiate how spending magnitude relates to credits and debits. Similarly, three interaction features were created between the scaled **Amount** and each one-hot encoded **Account Name** column to enable the models to learn whether spending patterns differ across account types, which would not be captured by the original features alone.

All feature engineering steps were implemented in a shared preprocessing pipeline (`data_loading.py`) to ensure consistency between training and testing data across all four models. The final feature set included original encoded features along with the engineered interaction terms.

2.2 Model Selection

Logistic Regression: As a linear classifier, logistic regression was selected because of its simplicity, interpretability, and strong performance on high-dimensional datasets. It models the probability of a data point belonging to a particular class using a logistic function (sigmoid), which makes it well-suited for multiclass classification. Logistic regression performs well when there is a linear relationship between the features and the target variable, which is what we aim to explore by employing this model.

Support Vector Machine (SVM): SVM was selected due to its effectiveness in high-dimensional feature spaces, which is relevant for this dataset after one-hot encoding of categorical variables and the inclusion

of engineered features. SVM aims to find an optimal decision boundary that maximizes the margin between classes, allowing it to generalize well to unseen data. In this project, a linear SVM was used, which is well-suited for high-dimensional data and provides efficient training while maintaining strong performance. This model is particularly useful for handling cases where classes are not easily separable and serves as a strong comparison to other linear and nonlinear models.

Random Forest: Given how it handles noisy and imbalanced data, non-linear decision boundaries, and overfitting, Random Forest was selected as one of the models for this dataset. As an ensemble method, Random Forest uses multiple decision trees to make predictions while reducing overfitting, resulting in better generalization and stable performance. Even in situations where the data is slightly imbalanced, the model remains reasonably effective. Additionally, this model works well on classes that might overlap in the feature space or in situations where categories are not always linearly separable.

K-Nearest Neighbors (KNN): KNN was selected because it is effective for classification problems where similar data points can belong to the same class. It classifies a data point based on the labels of its nearest neighbors in the feature space. This algorithm works well with datasets that have clear local patterns and can serve as a strong baseline model for comparison against more complex algorithms. KNN is easy to interpret and predictions can be explained by examining the nearest training examples.

2.3 Training Setup

To handle the class imbalance of the target variable, each model was trained and evaluated under two configurations: (1) a baseline model with no class weighting, and (2) a model using balanced class weights. The baseline configuration treats all classes equally, which can lead to bias toward majority classes but may increase accuracy since the dataset is significantly unbalanced. The balanced configuration assigns higher weights to minority classes and lower weights to majority classes, which allows the model to emphasize underrepresented categories. This modifies the penalty applied to misclassified samples during training and encourages the model to better learn minority class patterns without actually altering the original data distribution.

The dataset was split into training and testing sets using an 80/20 split. Hyperparameter tuning for each model was conducted using cross-validation on the training set, while the test set was held out for final evaluation to ensure unbiased results. Each model utilized a shared data loading script (`data_loading.py`) which ensured consistency for preprocessing and identical data splits across the four models.

2.4 Hyperparameter Tuning

Logistic Regression: Hyperparameter tuning was conducted using 5-fold cross-validation on the training set. The parameters evaluated were regularization strength ($C=0.001, 0.005, 0.01, 0.05, 1, 5, 10$), penalty type (L2 vs. L1), solver (saga vs. lbfgs), and class weighting (balanced vs. none). For each configuration, cross-validation accuracy and macro F1-score were computed and averaged across 5 folds using a maximum of 2,000 iterations. The best results of the hyperparameter tuning, which are the configurations that maximized the Macro F1-score, can be found in the table below:

Model	Penalty	Solver	Class Weight	Best C	Accuracy	Macro F1
Baseline Logistic Regression	L2	lbfgs	None	0.001	0.3689	0.1985
Balanced Logistic Regression	L2	lbfgs	Balanced	0.001	0.2185	0.1851

Support Vector Machine (SVM): Hyperparameter tuning was performed using 3-fold cross-validation on the training data. In this approach, the dataset was divided into three subsets, where the model was trained on two subsets and validated on the remaining subset. This process was repeated three times, and the results were averaged to evaluate model performance. A linear SVM was used, and tuning focused on the regularization parameter C, which controls the trade-off between maximizing the margin and minimizing classification error. Smaller values of C encourage a larger margin with more regularization, and larger values of C allow the model to fit the training data more closely.

The values tested for C were 0.001, 0.005, 0.01, 0.05, 0.1, 1, 5, and 10. For each value, cross-validation accuracy and weighted F1-score were computed, with macro F1-score also reported to evaluate performance across categories equally. Weighted F1-score was used as the primary evaluation metric to account for differences in the distribution of the categories. The results can be found in the table below:

SVM Model	Class Weight	Best C	Accuracy	Weighted F1	Macro F1
Baseline SVM	None	0.01	0.3700	0.3069	0.1919
Balanced SVM	Balanced	0.05	0.3673	0.3204	0.2055

Random Forest: Hyperparameter tuning was accomplished using Randomized Search and 3-fold cross-validation on the training set. Utilizing Randomized Search, different combinations of hyperparameters were randomly sampled and then evaluated using cross-validation. Based on the average cross-validated weighted F1 score, the best combination was then selected.

The hyperparameters tuned included: n_estimators, max_depth, min_samples_split, min_samples_leaf, and max_features. For each hyperparameter combination, the training data was split into three folds, modeled on two folds, and then validated on the remaining fold. This process was repeated three times and then the average of the scores was taken.

The table below shows the results of the trained Random Forest configurations:

Random Forest Model	Class Weight	Best Parameters	Accuracy	Weighted F1	Macro F1
---------------------	--------------	-----------------	----------	-------------	----------

Baseline Tuned	None	{'n_estimators': 300, 'min_samples_split': 5, 'min_samples_leaf': 4, 'max_features': 'log2', 'max_depth': 50}	0.3677	0.34	0.23
Balanced Random Forest	Balanced	Default	0.3478	0.34	0.23

K-Nearest Neighbors (KNN): Hyperparameter tuning was performed using Grid Search with 2-fold cross-validation on the training set. This allowed multiple parameter combinations to be tested systematically, with each model configuration evaluated using the weighted F1-score. The F1-score was selected as the main metric due to the imbalanced dataset across the larger and smaller classes.

The hyperparameters tuned for KNN were n_neighbors, weights, and p. The values tested for n_neighbors included 3, 5, 7, and 11. Uniform weighting and distance-based weighting were tested for weights. The values tested for p included 1 and 2, where p = 1 represented the Manhattan distance and p = 2 represented Euclidean distance under the Minkowski metric. This allowed for Grid Search to test 16 different hyperparameter combinations, resulting in a total of 32 fits with 2-fold cross-validation.

KNN Model	Class Weight	Best Parameters	Accuracy	Weighted F1	Macro F1
Baseline KNN	None	n_neighbors: 5, weights: uniform, p: 2	0.3399	0.3108	0.2051
Tuned KNN	None	n_neighbors: 7, weights: uniform, p: 1	0.3488	0.3208	0.2107

3. Results:

3.1 Evaluation Metrics

Given the objective of financial transaction classification and the characteristics of the dataset, multiple evaluation metrics were used to assess model performance. The primary metrics considered were precision, recall, and F1-score, along with overall accuracy and runtime performance.

Precision and recall were examined for each category to better understand model behavior. Precision measures the proportion of correct predictions among all predicted instances of a category, making it useful for evaluating the quality of the model's predictions. Recall measures the proportion of actual instances that were correctly identified, allowing us to assess how well the model captures relevant cases. Together, these metrics provide insight into both prediction accuracy and coverage.

Due to the presence of class imbalance in the dataset, the F1-score was selected as the primary evaluation metric. For example, certain categories appear much more frequently than others, while some categories

have relatively few examples. In such cases, overall accuracy may be misleading, as a model could achieve high accuracy by favoring more frequent categories. The F1-score balances precision and recall, providing a more reliable measure of performance across all categories. In this project, weighted F1-score was used as the main metric to account for differences in category distribution, while macro F1-score was also reported to evaluate performance equally across categories.

In addition to predictive performance, runtime efficiency was evaluated for each model. Both training time and prediction time were measured to analyze the trade-off between computational cost and model performance. This allows for comparison not only in terms of accuracy, but also in terms of practicality for real-world financial applications.

3.2 Model Performance

Logistic Regression: The performance of the baseline and balanced logistic regression models can be found in the figures below.

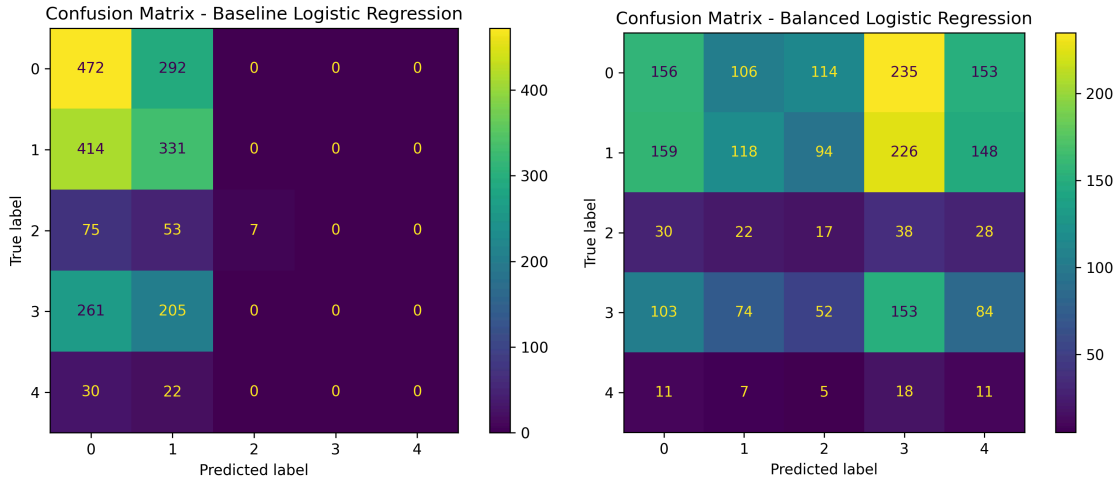
Figure 2.1. Baseline Logistic Regression Classification Report

	Precision	Recall	F1-Score	Support
0	0.34	0.204	0.255	764.0
1	0.361	0.158	0.22	745.0
2	0.06	0.126	0.082	135.0
3	0.228	0.328	0.269	466.0
4	0.026	0.212	0.046	52.0
accuracy	0.21	0.21	0.21	0.0
macro avg	0.203	0.206	0.174	2162.0
weighted avg	0.298	0.21	0.23	2162.0

Figure 2.2. Balanced Logistic Regression Classification Report

	Precision	Recall	F1-Score	Support
0	0.377	0.618	0.468	764.0
1	0.367	0.444	0.402	745.0
2	1.0	0.052	0.099	135.0
3	0.0	0.0	0.0	466.0
4	0.0	0.0	0.0	52.0
accuracy	0.375	0.375	0.375	0.0
macro avg	0.349	0.223	0.194	2162.0
weighted avg	0.322	0.375	0.31	2162.0

Figure 2.3. Confusion Matrix for Baseline (left) and Balanced (right) Logistic Regression



Support Vector Machine (SVM): The performance of the baseline and balanced SVM models can be found in the figures below.

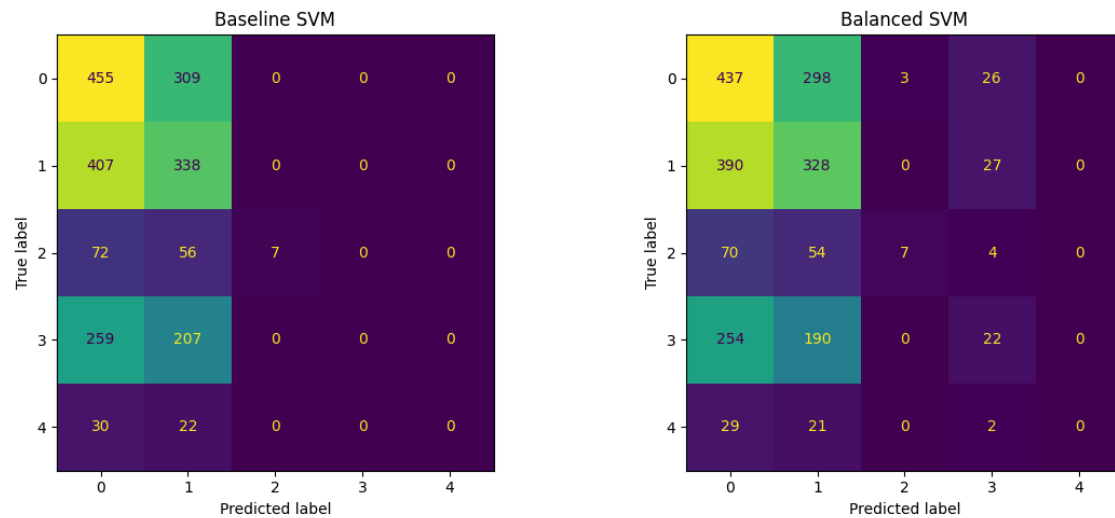
Figure 3.1. Baseline SVM Classification Report

	Precision	Recall	F1-Score	Support
0	0.372	0.596	0.458	764.0
1	0.363	0.454	0.403	745.0
2	1.0	0.052	0.099	135.0
3	0.0	0.0	0.0	466.0
4	0.0	0.0	0.0	52.0
accuracy	0.37	0.37	0.37	0.37
macro avg	0.347	0.22	0.192	2162.0
weighted avg	0.319	0.37	0.307	2162.0

Figure 3.2. Balanced SVM Classification Report

	Precision	Recall	F1-Score	Support
0	0.37	0.572	0.45	764.0
1	0.368	0.44	0.401	745.0
2	0.7	0.052	0.097	135.0
3	0.272	0.047	0.08	466.0
4	0.0	0.0	0.0	52.0
accuracy	0.367	0.367	0.367	0.367
macro avg	0.342	0.222	0.206	2162.0
weighted avg	0.36	0.367	0.32	2162.0

Figure 3.3. Confusion Matrix for Baseline (left) and Balanced (right) SVM



Random Forest: The performance of the baseline and balanced Random Forest models can be found in the figures below.

Figure 4.1. Baseline Random Forest and Balanced Random Forest Classification Reports

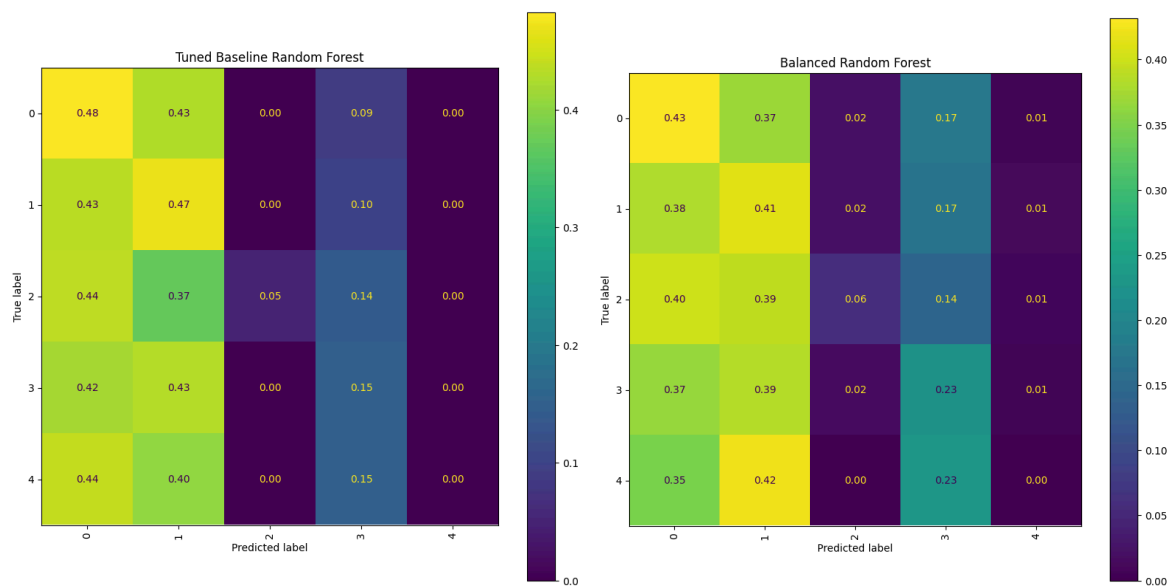
Baseline Tuned RF Classification Report

	Precision	Recall	F1-Score	Support
0	0.387	0.49	0.432	764.0
1	0.364	0.463	0.408	745.0
2	1.0	0.052	0.099	135.0
3	0.289	0.15	0.198	466.0
4	0.0	0.0	0.0	52.0
accuracy	0.368	0.368	0.368	0.368
macro avg	0.408	0.231	0.227	2162.0
weighted avg	0.387	0.368	0.342	2162.0

Balanced RF Classification Report

	Precision	Recall	F1-Score	Support
0	0.385	0.432	0.407	764.0
1	0.365	0.415	0.388	745.0
2	0.17	0.059	0.088	135.0
3	0.266	0.225	0.244	466.0
4	0.0	0.0	0.0	52.0
accuracy	0.348	0.348	0.348	0.348
macro avg	0.237	0.226	0.225	2162.0
weighted avg	0.33	0.348	0.336	2162.0

Figure 4.2. Confusion Matrix for Baseline (left) and Balanced (right) Random Forest



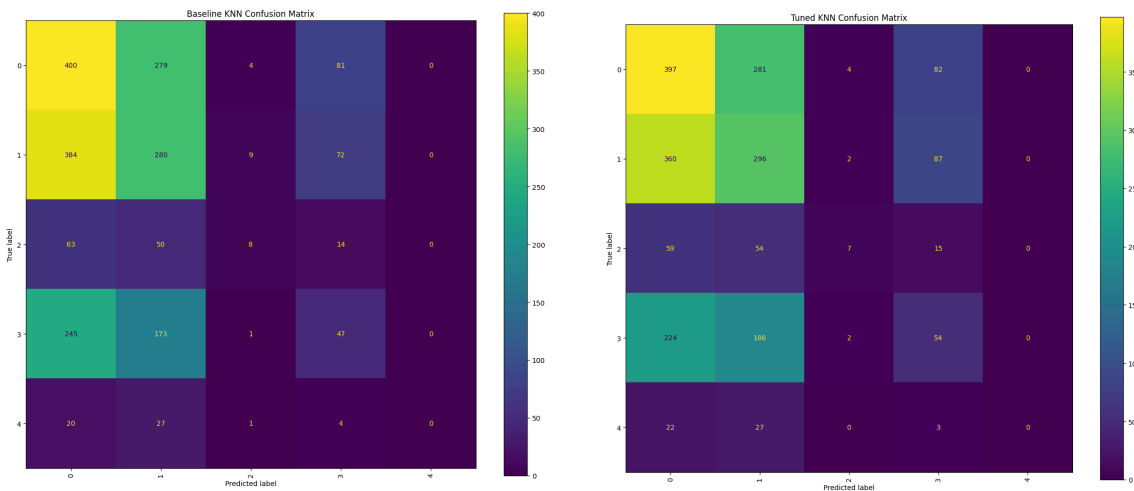
K-Nearest Neighbors: The performance of the baseline and tuned k-nearest neighbors models can be found in the figures below:

Figure 5.1. Baseline KNN Classification Report (Left) and Tuned KNN Classification Report (Right)

	Precision	Recall	F1-Score	Support
0	0.373823	0.519634	0.434830	764.000000
1	0.350711	0.397315	0.372561	745.000000
2	0.466667	0.051852	0.093333	135.000000
3	0.224066	0.115880	0.152758	466.000000
4	0.000000	0.000000	0.000000	52.000000
accuracy	0.348751	0.348751	0.348751	0.348751
macro avg	0.283053	0.216936	0.210697	2162.000000
weighted avg	0.330386	0.348751	0.320793	2162.000000

	Precision	Recall	F1-Score	Support
0	0.359712	0.523560	0.426439	764.000000
1	0.346106	0.375839	0.360360	745.000000
2	0.347826	0.059259	0.101266	135.000000
3	0.215596	0.100858	0.137427	466.000000
4	0.000000	0.000000	0.000000	52.000000
accuracy	0.339963	0.339963	0.339963	0.339963
macro avg	0.253848	0.211903	0.205098	2162.000000
weighted avg	0.314567	0.339963	0.310814	2162.000000

Figure 5.2. Baseline KNN Confusion Matrix (Left) & Tuned KNN Confusion Matrix (Right)



4. Discussion:

4.1 Model Comparison

As mentioned previously, the primary evaluation metrics used were F1-score, precision, recall, accuracy and runtime performance. Across all models, performance varies depending on how well each method was able to capture patterns within the feature space.

Logistic Regression and SVM showed similar overall performance, with both models achieving moderate accuracy and weighted F1-scores. These models were effective at identifying the more frequent categories but struggled with less represented ones. The use of class weighing improved recall for smaller categories but often resulted in a slight decrease in overall accuracy, reflecting the trade-off between balanced performance and overall correctness.

Random Forest demonstrated slightly improved performance in capturing more complex relationships between features due to its ability to model nonlinear patterns. Additionally, the model is robust to noise, which is helpful in cases of imbalanced datasets. However, the improvement was not substantial, as the model's Macro F1 score remained the same across all trained versions, even after tuning. This suggests that the available features may still limit the model's ability to distinguish between categories.

On the other hand, K-Nearest Neighbors (KNN) performed comparably but was more sensitive to the feature representation and scaling. While it was able to capture local patterns, its overall performance remained similar to the other models, indicating that no single model significantly outperformed the others.

In terms of runtime performance, simpler models such as Logistic Regression and SVM were more efficient, while models like Random Forest required more computation due to their complexity.

4.2 Challenges

A major challenge encountered during this project was deciding which features should be used to train the models. Initially, the models were trained using the cleaned dataset with the original **Category** labels provided in the data. However, it was discovered that the **Description** and **Category** columns were not properly aligned, and that the majority of the dataset (the last 10,000 rows) appeared to be synthetically generated with inconsistent category assignments that did not correspond to the transaction descriptions. As a result, the models were unable to learn meaningful relationships, leading to poor performance.

To address this issue, attention was shifted to the **Description** column, which contained clear keywords that could be mapped to transaction categories. While this approach improved model performance, it also made the classification task overly simplistic. The models relied heavily on keyword matching rather than learning broader patterns, resulting in unrealistically high performance.

To create a more realistic evaluation, the **Description** column was removed from the shared preprocessing pipeline. Instead, models were trained using features derived from transactional and temporal attributes,

including **Transaction Type**, **Day of Week**, **Month**, and a scaled version of the transaction **Amount**. Additionally, one-hot encoded account features and interaction features, such as the product of amount and transaction type, were included to capture more complex relationships in the data.

4.3 Key Insights

One key insight from this project is that the choice of features had a significant impact on model performance. When the models were trained using only the **Description** column, the classification became overly simplistic, as the models relied directly on keyword matching. However, once the **Description** column was removed from the preprocessing pipeline, the models were required to learn from broader transaction characteristics, including transaction type, temporal features, and account information. This increased the difficulty of the task, but resulted in a more realistic and meaningful evaluation of model performance.

Another key insight is in the effect of class imbalance on model performance across categories. Categories with more data, such as **Bills & Payments** and **Food & Dining**, were easier for the models to predict accurately. In contrast, the categories with fewer examples, such as **Transportation**, were more difficult to classify. This highlights the importance of using evaluation metrics beyond just accuracy, such as F1-score, to better assess how well each model performs across all categories.

4.4 Trade-offs

A major trade-off in this project was between achieving high performance and maintaining a realistic evaluation. When the **Description** column was used as a feature, the models achieved high performance by relying on keyword-based patterns. However, this did not reflect true generalization. Removing the **Description** column resulted in lower performance metrics, but provided a more accurate representation of how well the models could learn general transaction behavior.

Another important trade-off involved model complexity, interpretability, and performance. Logistic Regression and SVM models are simpler models that are easier to interpret, but they are limited in their ability to capture complex relationships in the data. In comparison, Random Forest was able to model nonlinear patterns and achieved slightly better performance overall, but at the cost of reduced interpretability. KNN was also intuitive and easy to understand; however, its performance depended heavily on feature scaling and distance relationships, making it less reliable when the categories were not clearly separable.

5. Conclusion:

5.1 Summary

In this project, we developed a supervised machine learning system to classify personal financial transactions. Using the Personal Transactions dataset, the data was cleaned and prepared through feature engineering, encoding, and scaling, which allowed us to create a more meaningful target dataset for model training.

Four models were trained and evaluated: Logistic Regression, Support Vector Machine (SVM), Random Forest, and K-Nearest Neighbors (KNN). Each model was assessed using accuracy, precision, recall, weighted F1-score, and macro F1-score to account for class imbalance. Among these models, Random Forest achieved the strongest overall performance based on its weighted F1-score and accuracy. However, the other models provided valuable comparisons in terms of interpretability, computational efficiency, and behavior across different categories.

Overall, the results demonstrate that while more complex models can capture nonlinear relationships more effectively, simpler models still offer competitive performance and are easier to interpret, highlighting the importance of balancing model complexity with practical considerations.

5.2 Future Work

Future improvements could focus on enhancing the dataset and feature representation to improve model performance. One potential example of this is applying natural language processing techniques on transaction descriptions. This would allow models to extract meaningful information such as merchant names, keywords, and spending patterns without relying on direct keyword matching. Additionally, incorporating features such as transaction frequency or the user spending behavior could also provide deeper insights and improve classification accuracy.

Another area for improvement is addressing the class imbalance in the dataset. Since categories like **Transportation** and **Income** contain relatively fewer examples, the models struggled to classify them consistently. Collecting more balanced transaction data or applying resampling techniques could help improve recall and F1-scores for these unrepresented categories.

Overall, this project contributes to the development of automated financial categorization systems, which can support applications such as budgeting, expense tracking, and personal financial planning.

6. Individual Contributions:

- **Aleksa Ocampo:** Developed and trained the Random Forest model. Collaborated on model evaluation by computing performance metrics and contributing to the comparison of model performance across all models. Contributed to writing and analysis in the Discussion section.
- **Alisa Crowe:** Performed data preprocessing, including data cleaning and feature engineering. Developed and trained the Logistic Regression model. Contributed to writing the Methodology (Data Preprocessing and Model Selection) sections.
- **Angela Lee:** Developed and trained the Support Vector Machine (SVM) model. Collaborated on model evaluation by computing performance metrics and contributing to the comparison of model performance across all models. Contributed to writing and analysis in the Results, Discussion, and Conclusion sections, and wrote the Abstract and Introduction sections.
- **Brianna Sengchan:** Performed data preprocessing, including data cleaning and encoding of the target variable. Developed and trained the K-Nearest Neighbors (KNN) model. Contributed to writing the Conclusion and Future Work sections and assisted in overall analysis.

7. References:

DAVID, S. N. (2024). *Personal transactions Data*. Kaggle.com.

<https://www.kaggle.com/datasets/shyakanobledavid/personal-transactions-userid-new-transaction-s/data>

OpenAI. (2025). *ChatGPT*. ChatGPT; OpenAI. <https://chatgpt.com/>

Pandas. (2018). *Python Data Analysis Library*. Pydata.org. <https://pandas.pydata.org/>

Ramya Pintchy. (2020). *Personal Finance Data*. Kaggle.com.

<https://www.kaggle.com/datasets/ramyapintchy/personal-finance-data>

Scikit-learn. (2024). *scikit-learn: Machine Learning in Python*. Scikit-Learn.org.

<https://scikit-learn.org/stable/>

Shafi, A. (2018, May 16). Random Forest Classification in Python With Scikit-Learn: Step-by-Step Guide (with Code Examples). Datacamp.Com; DataCamp.

<https://www.datacamp.com/tutorial/random-forests-classifier-python>